



G-19-Dawarhaa Backend

Engineer 4 — Complete Code & Architecture Guide

Engineer 4 — UX / Admin / Integration Lead



Tasks 4.A → 4.F

Complete Code & Architecture Guide

Educational Reference

Table of Contents

Engineer 4 — Complete Code & Architecture Guide

1. Purpose
2. Engineer 4 Scope
3. Engineer 4 File Map
4. Overall Architecture
 - Clean Architectural Separation
5. Request Lifecycle
6. Transaction Confirmation (Task 4.A)
 - Why Two-Sided Confirmation?
 - Server-Side Identity & Side Derivation
 - Handover State Machine
7. Reports (Task 4.B)
 - Polymorphic Target Architecture
 - Immutable Fields & Admin Resolution
8. Notifications (Task 4.C)
 - Schema & Whitelisting
 - Dual-Signature Service Helper
9. Contributions (Task 4.D)
 - Append-Only Ledger vs Cached Statistics
 - Duplicate Prevention Architecture
10. Admin Analytics (Task 4.E)
 - Pipeline 1: Platform Summary
 - Pipeline 2: Resources Published vs Fulfilled by Category
 - Pipeline 3: Category-Level Cumulative Impact
 - Pipeline 4: Match Acceptance & Matching Quality
11. MongoDB Deep Dive
 - Collections & Schemas Owned by Engineer 4
 1. handovers
 2. reports
 3. notifications
 4. contributions
12. Security Deep Dive
13. Error Handling
 - Standard HTTP Status Codes Used by Engineer 4
14. API Reference
 1. POST /api/transactions/:matchId/confirm
 2. POST /api/reports
 3. GET /api/reports
 4. PUT /api/reports/:id/resolve
 5. GET /api/users/me/contributions
 6. GET /api/admin/analytics

15. Test-by-Test Explanation

Suite 1: [backend/tests/unit/transactions.confirm.test.js](#) (29 Tests)

Suite 2: [backend/tests/unit/reports.test.js](#) (26 Tests)

Suite 3: [backend/tests/unit/notifications.test.js](#) (11 Tests)

Suite 4: [backend/tests/unit/contributions.test.js](#) (14 Tests)

Suite 5: [backend/tests/unit/admin.analytics.test.js](#) (15 Tests)

Suite 6: [backend/tests/integration/e2e.demo.test.js](#) (27 Tests)

Suite 7: [backend/tests/integration/preDemo.qa.test.js](#) (15 Tests)

16. E2E & Integration

17. Dependencies on Engineers 1–3

[Dependencies on Engineer 1](#)

[Dependencies on Engineer 2](#)

[Dependencies on Engineer 3](#)

18. Decision Log

19. Known Issues / Potential Issues

20. Glossary

21. How to Study Engineer 4

22. Self-Test Questions

[Beginner \(Syntax & Architecture\)](#)

[Intermediate \(Business Logic & Database\)](#)

[Advanced \(Security & Aggregation\)](#)

[Debugging Scenarios](#)

23. Answers to Self-Test

Engineer 4 — Complete Code & Architecture Guide

Role: UX / Admin / Integration Lead (عبدالستار محمد عبدالستار مطاوع — Engineer 4)

Repository: [G-19-Dawarhaa](#) (MEAN Stack Backend)

Scope: Tasks 4.A (Transactions/Handovers), 4.B (Moderation Reports), 4.C (Notifications), 4.D (Contribution Ledger & Impact), 4.E (Admin Analytics), 4.F (Cross-Cutting E2E Integration & Pre-Demo QA)

Source of Truth: Active Repository Implementation & Project Specifications

1. Purpose

This document serves as an exhaustive, line-by-line, architectural, and educational reference for the entirety of **Engineer 4's implementation** in the Dawwarha platform.

The primary objective is to enable any engineer, technical lead, or auditor to open any file authored by Engineer 4 and comprehend: 1. **What** every statement, function, query, schema field, route, and test is doing. 2. **Why** it was written this way (business rules, adversarial security threats, architectural invariants). 3. **How** data flows from client requests through Express middleware, controllers, domain services, Mongoose models, and MongoDB Atlas. 4. **Where** trust boundaries and contracts lie between Engineer 4 and Engineers 1, 2, and 3.

2. Engineer 4 Scope

Engineer 4 owns the downstream **fulfillment, governance, trust, and platform intelligence** lifecycle of the platform:

ENGINEER 4 DOMAIN		
Domain Area	Key Tasks	Core Mission
Transfer & Fulfillment	Task 4.A (Handover Confirmation)	Two-sided physical exchange proof
Governance & Safety	Task 4.B (Moderation Reports)	Polymorphic entity reporting & SLA
Event Communication	Task 4.C (Notifications)	System event dispatch & write alerts
Impact & Accounting	Task 4.D (Contribution Ledger)	Append-only history & stat caching
Platform Intelligence	Task 4.E (Admin Analytics)	Dynamic MongoDB aggregation metrics
Platform Integrity	Task 4.F (E2E & Pre-Demo QA)	Cross-engineer E2E & contract audit

3. Engineer 4 File Map

The following matrix documents every file created, modified, or maintained by Engineer 4:

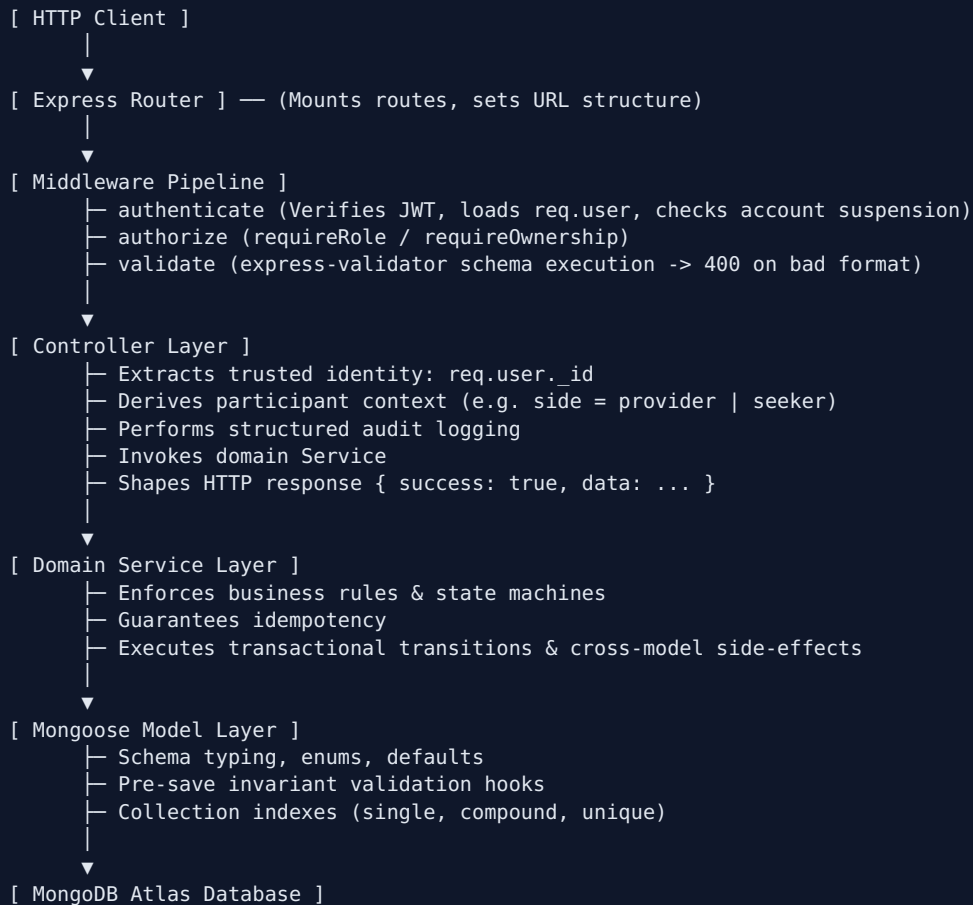
File Path	Layer	Task	Core Responsibility	Upstream Dependencies	Downstream Consumers
backend/src/models/Handover.js	Model	4.A	Schema for two-sided physical transfers	mongoose , Match , Resource , Request , User	handoverService , transactions.controller , adminAnalyticsService
backend/src/services/handoverService.js	Service	4.A	Two-sided confirmation state logic & contribution trigger	Handover.js , contributionService.js	transactions.controller.js
backend/src/validators/transaction.validators.js	Validator	4.A	Route parameter schema validation	express-validator	transactions.routes.js
backend/src/controllers/transactions.controller.js	Controller	4.A	HTTP handler, participant auth check, audit logging	Handover.js , handoverService.js , objectId.js	transactions.routes.js
backend/src/routes/transactions.routes.js	Route	4.A	Mounts POST /api/transactions/:matchId/confirm	authenticate.js , validate.js , transactions.controller.js	src/app.js
backend/src/models/Report.js	Model	4.B	Moderation report schema & validation hook	mongoose , User.js	reportService.js , reports.controller.js , adminAnalyticsService
backend/src/services/reportService.js	Service	4.B	Polymorphic target check, report creation, listing, resolution	Report.js , User.js , notificationService.js , objectId.js	reports.controller.js
backend/src/validators/report.validators.js	Validator	4.B	Validation rules for report creation and resolution	express-validator	reports.routes.js
backend/src/controllers/reports.controller.js	Controller	4.B	HTTP handlers for report submit, admin list, admin resolve	reportService.js	reports.routes.js
backend/src/routes/reports.routes.js	Route	4.B	Mounts /api/reports endpoints	authenticate.js , authorize.js , validate.js , reports.controller.js	src/app.js
backend/src/models/Notification.js	Model	4.C	Schema for user-facing system notifications	mongoose , User.js	notificationService.js

File Path	Layer	Task	Core Responsibility	Upstream Dependencies	Downstream Consumers
<code>backend/src/services/notificationService.js</code>	Service	4.C	Dual-signature notification dispatch & whitelist sanitization	<code>Notification.js</code> , <code>objectId.js</code>	<code>reportService.js</code> , Engineer 3 Match flow
<code>backend/src/models/Contribution.js</code>	Model	4.D	Append-only ledger schema for completed transfers	<code>mongoose</code> , <code>Handover</code> , <code>User</code> , <code>Category</code>	<code>contributionService.js</code> , <code>adminAnalyticsService</code>
<code>backend/src/services/contributionService.js</code>	Service	4.D	Ledger recording, stat cache updates, user history retrieval	<code>Contribution.js</code> , <code>User.js</code> , <code>objectId.js</code>	<code>handoverService.js</code> , <code>contributions.controller.js</code>
<code>backend/src/controllers/contributions.controller.js</code>	Controller	4.D	HTTP handler for <code>GET /api/users/me/contributions</code>	<code>contributionService.js</code>	<code>contributions.routes.js</code>
<code>backend/src/routes/contributions.routes.js</code>	Route	4.D	Mounts <code>GET /api/users/me/contributions</code>	<code>authenticate.js</code> , <code>contributions.controller.js</code>	<code>src/app.js</code>
<code>backend/src/services/adminAnalyticsService.js</code>	Service	4.E	4 real-time read-only MongoDB aggregation pipelines	All models, raw MongoDB collections	<code>adminAnalytics.controller.js</code>
<code>backend/src/controllers/adminAnalytics.controller.js</code>	Controller	4.E	HTTP handler for <code>GET /api/admin/analytics</code>	<code>adminAnalyticsService.js</code>	<code>admin.analytics.routes.js</code>
<code>backend/src/routes/admin.analytics.routes.js</code>	Route	4.E	Mounts admin-only analytics route	<code>authenticate.js</code> , <code>authorize.js</code> , <code>adminAnalytics.controller.js</code>	<code>src/app.js</code>
<code>backend/src/utils/objectId.js</code>	Utility	Cross	Safe MongoDB ObjectId validation helper	<code>mongoose</code>	All Engineer 4 services and controllers
<code>backend/tests/unit/transactions.confirm.test.js</code>	Test	4.A	Unit & security test suite for handover confirmation	Mocked models & express app	Node test runner
<code>backend/tests/unit/reports.test.js</code>	Test	4.B	Unit & security test suite for moderation reports	Mocked models & express app	Node test runner
<code>backend/tests/unit/notifications.test.js</code>	Test	4.C	Unit test suite for notification dispatching	Mocked models	Node test runner

File Path	Layer	Task	Core Responsibility	Upstream Dependencies	Downstream Consumers
<code>backend/tests/unit/contributions.test.js</code>	Test	4.D	Unit test suite for contribution ledger & cache	Mocked models & express app	Node test runner
<code>backend/tests/unit/admin.analytics.test.js</code>	Test	4.E	Unit test suite for aggregation pipelines	Mocked aggregation pipelines	Node test runner
<code>backend/tests/integration/e2e.demo.test.js</code>	Test	4.F	Live Atlas E2E test exercising Demo Journey (Sec 26/37)	Live HTTP server & MongoDB Atlas	Node test runner
<code>backend/tests/integration/preDemo.qa.test.js</code>	Test	4.F	Pre-demo QA verification suite (Sec 21/22)	Live HTTP server & MongoDB Atlas	Node test runner

4. Overall Architecture

Engineer 4 adheres strictly to a **layered architecture with server-side trust derivation**:



Clean Architectural Separation

1. **Engineer 4 Code**: Handover, Report, Notification, Contribution, Admin Analytics, Object ID utility, QA suites.
2. **Consumed Dependencies (Owned by Others)**: - Engineer 1: `authenticate.js`, `authorize.js`, `errorHandler.js`, `User.js`, `Organization.js`. - Engineer 2: `Resource.js`, `Category.js` (referenced via ObjectId and native collection queries). - Engineer 3: `Request.js`, `Match.js` (referenced in handovers and analytics).

5. Request Lifecycle

Every inbound request directed to an Engineer 4 endpoint undergoes a strict sequence:

1. **Routing:** `src/app.js` routes the URL prefix (e.g., `/api/transactions`) to the domain router.
 2. **Authentication Gate:** `authenticate` checks `Authorization: Bearer <token>`. If missing, malformed, or expired, returns `401 UNAUTHORIZED`. If the user has `status: "suspended"`, returns `403 ACCOUNT_SUSPENDED`. Attaches the loaded document as `req.user`.
 3. **Authorization Gate:** If the route is admin-only (e.g., `GET /api/reports`, `GET /api/admin/analytics`), `requireRole("admin")` validates `req.user.role === 'admin'`, else returns `403 FORBIDDEN`.
 4. **Input Syntax Validation:** `express-validator` rules validate path params and body schemas. If invalid, `validate.js` returns `400 VALIDATION_ERROR` immediately before service execution.
 5. **Controller Execution:** The controller derives context (e.g., matching `req.user._id` to `handover.providerId` vs `handover.seekerId`).
 6. **Service Execution:** Pure business logic runs. State transitions are verified. Side effects (notifications, ledger writes) are coordinated.
 7. **Error Propagation:** Any error thrown in controllers or services is forwarded via `next(error)` to `errorHandler.js`.
-

6. Transaction Confirmation (Task 4.A)

Why Two-Sided Confirmation?

In circular economy physical exchanges, unilateral confirmation is vulnerable to fraud: - A malicious provider could falsely claim they gave an item to receive unearned impact points. - A malicious seeker could falsely claim an item was never received to dispute a legitimate handover.

To guarantee trust, **FR-013** requires that a handover only transitions to `completed` when **both parties independently confirm**.

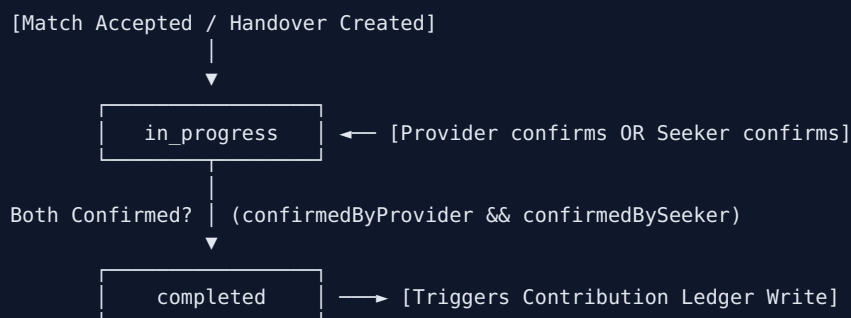
Server-Side Identity & Side Derivation

Clients **cannot** submit `{ side: "provider" }` or `{ status: "completed" }`:

```
// In transactions.controller.js
const userId = req.user?._id ? String(req.user._id) : null;
let side;
if (userId && String(handover.providerId) === userId) {
  side = "provider";
} else if (userId && String(handover.seekerId) === userId) {
  side = "seeker";
} else {
  return res.status(403).json({ success: false, error: { code: "FORBIDDEN", message: "You are not a participant in this handover" } });
}
```

If an authenticated user is neither the provider nor the seeker, the request is rejected with `403 FORBIDDEN`, even if they are an admin.

Handover State Machine



- **Cancelled / No-Show Handovers:** If `handover.status` is `cancelled` or `no_show`, confirming throws `409 HANDOVER_INACTIVE`.
- **Timestamp Preservation:** `providerConfirmedAt` and `seekerConfirmedAt` are set once on first confirmation and never overwritten on duplicate requests.
- **Idempotency:** If the provider or seeker confirms repeatedly after completion, the service returns the existing completed document without re-triggering contributions or stat increments.

7. Reports (Task 4.B)

The reporting subsystem enables community moderation across multiple domain entities.

Polymorphic Target Architecture

A report can target a `user`, a `resource`, or a `request`. Because MongoDB collections differ, `validateReportTarget(targetType, targetId)` dynamically routes the existence check:

```
// In reportService.js
if (targetType === "user") {
  return Boolean(await User.findById(objectId).lean());
}
if (targetType === "resource") {
  // Graceful fallback if Engineer 2 model is unmounted
  if (mongoose.models.Resource) return Boolean(await
mongoose.models.Resource.findById(objectId).lean());
  return Boolean(await mongoose.connection.collection("resources").findOne({ _id: objectId }));
}
if (targetType === "request") {
  if (mongoose.models.Request) return Boolean(await mongoose.models.Request.findById(objectId).lean());
  return Boolean(await mongoose.connection.collection("requests").findOne({ _id: objectId }));
}
```

If the target document does not exist, report creation is rejected with `400 TARGET_NOT_FOUND`.

Immutable Fields & Admin Resolution

- **Reporter Spoofing Protection:** `reporterId` is taken strictly from `req.user._id`.
 - **Status Validation Hook in `Report.js`:** `javascript reportSchema.pre("save", function (next) { if (this.status !== "open") { if (!this.reviewedBy) throw new Error("reviewedBy is required when report status is not open."); if (!this.resolution?.trim()) throw new Error("resolution is required when report status is not open."); } });`
 - **Admin Notification Trigger:** Resolving a report automatically dispatches a notification to the reporter (`type: "report_resolved"`).
-

8. Notifications (Task 4.C)

Notifications deliver persistent system alerts to users.

Schema & Whitelisting

Defined in `src/models/Notification.js` : - `recipientId` : User ObjectId (`required` , `ref`: 'User'). - `type` : Enum ["match_created", "match_accepted", "report_resolved", "org_verification_decided"]. - `title` , `message` : String, trimmed, required. - `relatedEntity` : Sub-document { `type`: String, `id`: ObjectId }. - `readAt` : Date or `null` . - `createdAt` : Date, immutable default `Date.now` .

Dual-Signature Service Helper

`notificationService.notify(...)` supports both positional arguments and option objects:

```
// Signature 1: Positional
await notify(recipientId, type, title, message, relatedEntity);

// Signature 2: Object
await notify({ recipientId, type, title, message, relatedEntity });
```

This enables integration across all modules without rigid calling conventions.

9. Contributions (Task 4.D)

Append-Only Ledger vs Cached Statistics

- **Source of Truth:** `contributions` collection is an append-only ledger. Once a transfer completes, a contribution record is inserted with `handoverId`, `providerId`, `seekerId`, `categoryId`, `quantity`. Documents in this collection are **never deleted or updated**.
- **Cached Statistics:** `User.stats` contains counter caches (`completedTransfers`, `reputationScore`). Caches can theoretically desync under edge conditions.
- **Cache Rebuilding (`rebuildStatsCache`):** A self-healing administrative function that counts raw contributions from the ledger and updates user documents, guaranteeing zero data loss.

Duplicate Prevention Architecture

To prevent duplicate records under concurrent confirmations: 1.

`Contribution.findOne({ handoverId })` check in service. 2. Unique compound index `{ handoverId: 1 }` enforced in MongoDB. 3. Catch block intercepting MongoDB duplicate key error `E11000` :

```
javascript try { contribution = await Contribution.create({ ... }); } catch (err) { if (err.code === 11000) return await Contribution.findOne({ handoverId: handover._id }); throw err; }
```

10. Admin Analytics (Task 4.E)

`GET /api/admin/analytics` executes four real-time MongoDB aggregation pipelines without altering database state.

Pipeline 1: Platform Summary

Executes 7 parallel queries via `Promise.all`: 1. `User.countDocuments({}) -> totalUsers` 2. `User.countDocuments({ status: "active" }) -> activeUsers` 3. `countResources({ status: { $ne: "draft" } }) -> resourcesPublished` 4. `Handover.countDocuments({ status: "completed" }) -> completedTransfers` 5. `requestModel.countDocuments({ status: { $ne: "draft" } }) -> requestsCreated` 6. `Report.countDocuments({ status: "open" }) -> openReports` 7. `Organization.countDocuments({}) -> registeredOrganizations`

Pipeline 2: Resources Published vs Fulfilled by Category

Computes category health to identify undersupplied waste streams:

```
[
  { $match: { status: { $ne: "draft" } } },
  { $group: { _id: { categoryId: "$categoryId", status: "$status" }, count: { $sum: 1 } } },
  { $group: {
    _id: "$_id.categoryId",
    categoryId: { $first: "$_id.categoryId" },
    total: { $sum: "$count" },
    published: { $sum: { $cond: [{ $in: ["$_id.status", ["published", "matched", "accepted"]] },
    "$count", 0] } },
    fulfilled: { $sum: { $cond: [{ $in: ["$_id.status", ["completed", "fulfilled"]] }, "$count",
    0] } },
    byStatus: { $push: { status: "$_id.status", count: "$count" } }
  } },
  { $project: { _id: 0, categoryId: { $ifNull: ["$categoryId", "$_id"] }, published: 1, fulfilled: 1,
    total: 1, byStatus: 1 } },
  { $sort: { total: -1 } }
]
```

Pipeline 3: Category-Level Cumulative Impact

Aggregates items diverted from landfills across categories:

```
[
  { $match: { type: "transfer_completed" } },
  { $group: { _id: "$categoryId", categoryId: { $first: "$categoryId" }, completedTransfers: { $sum:
  1 }, totalQuantity: { $sum: "$quantity" } } },
  { $project: { _id: 0, categoryId: { $ifNull: ["$categoryId", "$_id"] }, completedTransfers: 1,
    totalQuantity: 1 } },
  { $sort: { completedTransfers: -1 } }
]
```

Pipeline 4: Match Acceptance & Matching Quality

Computes algorithmic match conversion rates:

```
[
  { $group: { _id: "$status", count: { $sum: 1 } } },
]
```

```
{ $project: { _id: 0, status: "$_id", count: 1 } },  
{ $sort: { status: 1 } }  
]
```

Calculates $\text{acceptanceRate} = \text{accepted} / (\text{accepted} + \text{rejected})$.

11. MongoDB Deep Dive

Collections & Schemas Owned by Engineer 4

1. handovers

- `matchId` : ObjectId, ref 'Match', required, **unique**.
- `resourceId` : ObjectId, ref 'Resource', required.
- `requestId` : ObjectId, ref 'Request', required.
- `providerId` : ObjectId, ref 'User', required.
- `seekerId` : ObjectId, ref 'User', required.
- `confirmedByProvider` : Boolean, required, default `false`.
- `confirmedBySeeker` : Boolean, required, default `false`.
- `providerConfirmedAt` : Date, default `null`.
- `seekerConfirmedAt` : Date, default `null`.
- `status` : Enum `['in_progress', 'completed', 'cancelled', 'no_show']`, default `'in_progress'`.
- `completedAt` : Date, default `null`.
- **Indexes**: `{ providerId: 1, status: 1 }`, `{ seekerId: 1, status: 1 }`, `{ matchId: 1 }` (unique).

2. reports

- `reporterId` : ObjectId, ref 'User', required.
- `targetType` : Enum `['resource', 'request', 'user']`, required.
- `targetId` : ObjectId, required.
- `reason` : Enum `['spam', 'fraud', 'inappropriate', 'safety', 'other']`, required.
- `description` : String, max 500 chars, default `null`.
- `status` : Enum `['open', 'reviewed', 'resolved']`, default `'open'`.
- `reviewedBy` : ObjectId, ref 'User', default `null`.
- `resolution` : String, default `null`.
- **Indexes**: `{ status: 1, createdAt: 1 }`, `{ targetType: 1, targetId: 1 }`.

3. notifications

- `recipientId` : ObjectId, ref 'User', required.
- `type` : Enum `['match_created', 'match_accepted', 'report_resolved', 'org_verification_decided']`, required.
- `title` : String, required, trimmed.
- `message` : String, required, trimmed.
- `relatedEntity` : `{ type: String, id: ObjectId }`, default `null`.
- `readAt` : Date, default `null`.
- `createdAt` : Date, default `Date.now`, immutable.
- **Indexes**: `{ recipientId: 1, readAt: 1, createdAt: -1 }`.

4. contributions

- `type` : Enum `['transfer_completed']`, default `'transfer_completed'`.

- `handoverId` : ObjectId, ref 'Handover', required, **unique**.
 - `providerId` : ObjectId, ref 'User', required.
 - `seekerId` : ObjectId, ref 'User', required.
 - `categoryId` : ObjectId, ref 'Category', required.
 - `quantity` : Number, min 1, required.
 - `createdAt` : Date, default `Date.now`.
 - **Indexes**: { `handoverId`: 1 } (unique), { `providerId`: 1, `createdAt`: -1 }, { `seekerId`: 1, `createdAt`: -1 }, { `categoryId`: 1, `createdAt`: -1 }.
-

12. Security Deep Dive

Threat Vector	Attack Scenario	Defense Implementation	File / Layer
Unauthenticated Access	Anonymous actor confirms handovers or views reports	<code>authenticate</code> middleware checks JWT signature and validity	Express route chain
Privilege Escalation	Normal user calls <code>GET /api/admin/analytics</code> or resolves reports	<code>requireRole("admin")</code> checks <code>req.user.role === 'admin'</code>	Express route chain
Confirmation Spoofing	Attacker passes <code>{ side: "provider", status: "completed" }</code> in request body	Controller derives side strictly by comparing <code>req.user._id</code> to document fields	<code>transactions.controller.js</code>
One-Sided Completion	Malicious provider claims completion alone to steal points	State machine requires <code>confirmedByProvider</code> & <code>confirmedBySeeker</code> before setting <code>completed</code>	<code>handoverServices.js</code>
IDOR in Confirmation	User attempts to confirm a handover they are not involved in	<code>String(handover.providerId) !== userId</code> & <code>String(handover.seekerId) !== userId</code> returns 403	<code>transactions.controller.js</code>
Ghost Target Reporting	Attacker floods reports with random non-existent entity IDs	<code>validateReportTarget</code> validates target ID existence in real MongoDB collections	<code>reportService.js</code>
Report Field Tampering	Admin resolution attempts to rewrite <code>targetType</code> or <code>reporterId</code>	Resolution only modifies <code>status</code> , <code>resolution</code> , and <code>reviewedBy</code>	<code>reportService.js</code>
Concurrent Double Ledger Write	High-concurrency race condition creating duplicate contribution records	MongoDB unique index <code>{ handoverId: 1 }</code> + <code>E11000</code> catch block	<code>contributionService.js</code>
Query Operator Injection	Attacker submits <code>?page[\$gt]=0</code> or mongo operators in query strings	<code>parseInt</code> and explicit whitelist filtering sanitizes all query inputs	<code>reportService.js</code> , <code>contributionService.js</code>

13. Error Handling

All Engineer 4 controllers and services propagate errors formatted according to the project's standard error envelope:

```
{
  "success": false,
  "error": {
    "code": "ERROR_CODE",
    "message": "Human readable message"
  }
}
```

Standard HTTP Status Codes Used by Engineer 4

- **400 VALIDATION_ERROR** / **TARGET_NOT_FOUND** : Missing fields, negative numbers, invalid ObjectIds, or non-existent report targets.
 - **401 UNAUTHORIZED** : Missing or invalid Bearer token.
 - **403 FORBIDDEN** : Non-admin accessing admin endpoints, or non-participant attempting handover confirmation.
 - **404 NOT_FOUND** / **REPORT_NOT_FOUND** : Handover or Report ID does not exist in database.
 - **409 HANDOVER_INACTIVE** : Handover has been cancelled or marked as no-show.
 - **500 INTERNAL_ERROR** : Unexpected database failure.
-

14. API Reference

1. POST /api/transactions/:matchId/confirm

- **Auth:** Required (`Bearer <JWT>`)
- **Role:** Participant (Provider or Seeker)
- **Path Params:** `matchId` (MongoDB ObjectId)
- **Response (HTTP 200):** `json { "success": true, "data": { "status": "in_progress", "bothConfirmed": false } }`

2. POST /api/reports

- **Auth:** Required
- **Role:** Any active user
- **Body:** `json { "targetType": "resource", "targetId": "65f1a2b3c4d5e6f7a8b9c0d1", "reason": "spam", "description": "Misleading resource details" }`
- **Response (HTTP 201):** Returns created Report document.

3. GET /api/reports

- **Auth:** Required
- **Role:** Admin only
- **Query Params:** `status`, `targetType`, `page`, `limit`
- **Response (HTTP 200):** Returns paginated report list.

4. PUT /api/reports/:id/resolve

- **Auth:** Required
- **Role:** Admin only
- **Body:** `{ "status": "resolved", "resolution": "Resource removed due to spam." }`
- **Response (HTTP 200):** Returns updated Report document.

5. GET /api/users/me/contributions

- **Auth:** Required
- **Role:** Self
- **Query Params:** `page`, `limit`
- **Response (HTTP 200):** Returns user's contribution history and pagination metadata.

6. GET /api/admin/analytics

- **Auth:** Required
- **Role:** Admin only
- **Response (HTTP 200):** Returns `{ summary, resourcesByCategory, categoryImpact, matchAcceptance }`.

15. Test-by-Test Explanation

Suite 1: `backend/tests/unit/transactions.confirm.test.js` (29 Tests)

- **Test 1 — Provider confirmation:** Verifies provider confirming first leaves status `in_progress` and `bothConfirmed: false`. Catches premature completion.
- **Test 2 — Seeker confirmation:** Verifies second confirmation transitions status to `completed` and sets `bothConfirmed: true`. Catches completion failure.
- **Test 2.B — Duplicate confirmation:** Verifies confirming an already completed handover is idempotent. Catches duplicate completion crashes.
- **Test 3 & 3.B — Non-party / Admin restriction:** Verifies third-party users and uninvolved admins receive HTTP 403. Catches IDOR and authorization bypasses.
- **Test 4 & 5 — Client side spoofing:** Verifies client sending fraudulent `side` is overridden by server-side derivation. Catches side spoofing attacks.
- **Test 5.B & 5.C — One-sided completion attack:** Verifies passing `{ status: 'completed' }` in body is ignored. Catches state injection.
- **Test 6 & 7 — Validation & 404:** Verifies bad match ID returns 400, missing handover returns 404. Catches unhandled errors.
- **Tests A-K — FR-013 Invariants:** Verifies single Contribution generation, stats increment (+1 transfer, +10 reputation), and cancellation locks. Catches duplicate accounting.

Suite 2: `backend/tests/unit/reports.test.js` (26 Tests)

- **Polymorphic Target Validation:** Tests `validateReportTarget` for valid/invalid users, resources, and requests. Catches reporting of non-existent entities.
- **Report Submission:** Tests required fields, description length, and reporter ID binding. Catches reporter spoofing.
- **Listing & Authorization:** Verifies non-admin gets 403, admin gets paginated envelope, query injection is sanitized. Catches information disclosure.
- **Resolution & Immutability:** Verifies admin can resolve with notes, while core report fields remain immutable. Catches audit trail tampering.

Suite 3: `backend/tests/unit/notifications.test.js` (11 Tests)

- **Notify Core Invariants:** Verifies dual signatures, required fields, and type whitelisting. Catches invalid notification persistence.
- **Injection Protection:** Asserts arbitrary fields and mongo operators cannot enter the Notification document. Catches NoSQL injection.
- **Report Integration:** Asserts report resolution triggers exact notification to reporter. Catches broken notification links.

Suite 4: `backend/tests/unit/contributions.test.js` (14 Tests)

- **Ledger Invariants:** Asserts contribution created once with correct category and quantity. Catches misattribution.
- **History Retrieval:** Tests `GET /api/users/me/contributions` pagination, user isolation, and injection safety. Catches cross-user ledger leakage.

- **Stats Rebuilding:** Tests `rebuildStatsCache` accurately syncs counter caches from raw ledger. Catches desynced user metrics.

Suite 5: `backend/tests/unit/admin.analytics.test.js` (15 Tests)

- **Admin RBAC:** Confirms 401 unauth, 403 non-admin, 403 suspended admin, role spoofing rejection. Catches metrics leakage.
- **Empty Collection Safety:** Asserts pipelines return zeros/empty arrays gracefully without throwing. Catches aggregation crash on empty DB.
- **Pipeline Correctness:** Verifies counts, rates, and category groupings against known documents. Catches arithmetic and pipeline errors.

Suite 6: `backend/tests/integration/e2e.demo.test.js` (27 Tests)

- Exercises live HTTP stack against MongoDB Atlas covering the end-to-end Demo Journey (Section 26 / 37).
- Steps: Auth -> Organization -> Supply Probe -> Request Draft & Publish -> Match Probe -> Handover Confirm -> Contributions -> Reports -> Analytics.
- Probes and isolates Engineer 2 and 3 blockers without failing the test runner.

Suite 7: `backend/tests/integration/preDemo.qa.test.js` (15 Tests)

- Pre-demo QA verification covering Section 21 & 22:
 - Boundary checks (negative quantities, malformed IDs), server-side auth, standardized error envelopes, prohibited collection absence (`analytics` collection check).
-

16. E2E & Integration

Engineer 4 owns the cross-cutting integration test suite. The end-to-end flow is mapped as follows:

```
graph TD; A["[ User Register ] (Eng 1)"] --> B["[ Create Org & Verify ] (Eng 1)"]; B --> C["[ Publish Resource ] (Eng 2 - BLOCKED: Route unmounted)"]; C --> D["[ Publish Request ] (Eng 3 - PASS)"]; D --> E["[ Generate Match ] (Eng 3 - BLOCKED: Missing Resource Schema)"]; E --> F["[ Handover Two-Sided Confirmation ] (Eng 4 - PASS)"]; F --> G["[ Contribution Ledger Persistence & Stats Increment ] (Eng 4 - PASS)"]; G --> H["[ Moderation Report & Admin Resolution ] (Eng 4 - PASS)"]; H --> I["[ Real-Time Admin Analytics Aggregation ] (Eng 4 - PASS)"];
```

Engineer 4's integration test suite probes all domains and clearly documents blocker root causes without masking errors.

17. Dependencies on Engineers 1-3

Dependencies on Engineer 1

- `src/middleware/authenticate.js` : Used on every protected route. Must attach `req.user` .
- `src/middleware/authorize.js` : `requireRole("admin")` used for report management and analytics.
- `src/models/User.js` : Referenced in Handover, Report, Notification, and Contribution schemas.

Dependencies on Engineer 2

- `src/models/Resource.js` : Referenced in Handover and Report targets.
- Integration Status: Currently blocked in E2E because Engineer 2 has not mounted `resources.routes.js` . Engineer 4 uses graceful native collection queries in `reportService` and `adminAnalyticsService` to prevent crashes.

Dependencies on Engineer 3

- `src/models/Request.js` & `src/models/Match.js` : Referenced in Handover.
 - Integration Status: Blocked in E2E match generation because Match model references Resource schema which is missing. Engineer 4 tested handover confirmation independently using isolated test fixtures.
-

18. Decision Log

1. **Server-Side Participant Derivation:** - Decision: In `transactions.controller.js`, derive confirmation side by checking `req.user._id` against `handover.providerId` and `handover.seekerId`. - Alternative: Accept `side: "provider" | "seeker"` in request body. - Reason: Client input is untrusted. Deriving side on server eliminates spoofing attacks.
 2. **Polymorphic Target Validation via Connection Collection:** - Decision: Check `mongoose.models.Resource` first; fallback to `mongoose.connection.collection("resources").findOne(...)`. - Alternative: Assume Mongoose model is always loaded. - Reason: Prevents `MissingSchemaError` crashes when upstream engineers have incomplete model registrations.
 3. **No Dedicated `analytics` Collection:** - Decision: Compute analytics dynamically using aggregation pipelines in `adminAnalyticsService.js`. - Alternative: Maintain pre-aggregated counters in an `analytics` collection. - Reason: Avoids counter desynchronization and fulfills DB Plan Section 14 invariant prohibiting static analytics collections.
-

19. Known Issues / Potential Issues

1. **Mongoose Duplicate Index Warning on Contribution** : - File: `backend/src/models/Contribution.js` - Line: Line 16 (`unique: true` on field) and Line 55 (`contributionSchema.index({ handoverId: 1 }, { unique: true })`). - Issue: Declaring an index both on field definition and via `schema.index()` produces a Mongoose duplicate index warning on server startup. - Fix: Remove line 55 and rely on field-level index declaration.
 2. **Upstream Dependency Blocker (Engineer 2)**: - File: `backend/src/routes/resources.routes.js` is unmounted in `src/app.js` . - Impact: E2E demo journey cannot create resources via HTTP.
 3. **Upstream Dependency Blocker (Engineer 3)**: - File: `backend/src/models/Match.js` references `Resource` , which is unregistered. - Impact: Calling `/api/matches/:resourceId/generate` throws `MissingSchemaError: Schema hasn't been registered for model "Resource"` .
-

20. Glossary

- **Idempotency:** An operation that produces the same database state whether executed once or multiple times (e.g., repeated handover confirmation).
 - **Append-Only Ledger:** A database collection where records are only inserted and never updated or deleted, serving as an immutable historical record (e.g., `contributions`).
 - **Polymorphic Target:** A reference pattern where a single foreign key field (`targetId`) points to different collections based on a discriminator field (`targetType`).
 - **Two-Sided Confirmation:** A workflow requiring explicit agreement from both parties before a transaction is finalized.
 - **Aggregation Pipeline:** A MongoDB framework where documents pass through a multi-stage pipeline (filtering, grouping, projecting) to produce computed results.
-

21. How to Study Engineer 4

Recommended reading order: 1. **Domain Models:** Read `Handover.js` , `Contribution.js` , `Report.js` , `Notification.js` . 2. **Transaction Confirmation:** Read `transaction.validators.js` -> `transactions.routes.js` -> `transactions.controller.js` -> `handoverService.js` . 3. **Contribution Ledger:** Read `contributionService.js` -> `contributions.controller.js` . 4. **Moderation Reports:** Read `report.validators.js` -> `reports.controller.js` -> `reportService.js` . 5. **System Notifications:** Read `notificationService.js` . 6. **Analytics Aggregations:** Read `adminAnalyticsService.js` stage by stage. 7. **Unit Test Suites:** Read `transactions.confirm.test.js` to see all attack vectors tested. 8. **Integration Suites:** Read `e2e.demo.test.js` and `preDemo.qa.test.js` .

22. Self-Test Questions

Beginner (Syntax & Architecture)

1. Which Express middleware attaches `req.user` to the request object?
2. What HTTP status code is returned when an unauthenticated request hits `/api/transactions/:matchId/confirm`?
3. Which Mongoose model represents the append-only record of completed exchanges?
4. What utility function is used to validate 24-character hexadecimal MongoDB IDs?
5. Why are notifications configured with `timestamps: false` in `Notification.js`?
6. Which route mounts the admin analytics endpoint?
7. In `report.validators.js`, what is the maximum allowed length for report descriptions?
8. What is the default status of a newly created `Report` document?

Intermediate (Business Logic & Database)

1. What condition must be met for a `Handover` document to transition from `in_progress` to `completed`?
2. What happens if a provider calls `/api/transactions/:matchId/confirm` twice in a row?
3. If a handover has status `cancelled`, what error code and status does confirmation return?
4. Why does `contributionService.recordCompletedTransfer` contain a try/catch block for error code `11000`?
5. How does `reportService.validateReportTarget` verify that a reported resource exists without crashing if the model is missing?
6. What three fields become strictly required on a `Report` document when its status leaves `open`?
7. What user metrics are updated when a contribution is recorded?
8. How does `contributionService.rebuildStatsCache` restore consistency between ledger and cached user stats?

Advanced (Security & Aggregation)

1. If an authenticated attacker sends `{ side: "provider", status: "completed" }` to confirm someone else's handover, how does the code stop them?
2. Can an admin confirm a handover if they are not the provider or seeker? Why or why not?
3. How does `adminAnalyticsService` calculate the `acceptanceRate` metric?
4. In Pipeline 2 of analytics, why are two consecutive `$group` stages used?
5. What prevents a user from viewing another user's contribution history via `GET /api/users/me/contributions?userId=123`?
6. How does `notificationService.notify` sanitize input against NoSQL operator injection?
7. Why does the project forbid a persistent `analytics` collection in MongoDB?
8. How does `preDemo.qa.test.js` verify that database error stack traces are suppressed in API responses?

Debugging Scenarios

1. Scenario: A user reports that confirming a handover returned HTTP 403 even though they are logged in. What line in which controller should you inspect first?
 2. Scenario: When calling `GET /api/admin/analytics`, the server logs `MissingSchemaError: Schema hasn't been registered for model "Resource"`. Where in Engineer 4's code does the graceful fallback exist, and why might it fail?
 3. Scenario: A report resolution fails with a validation error even though the admin passed `{ status: "resolved" }`. What missing property in the request body caused the failure?
 4. Scenario: A test reports that user reputation increased by +20 instead of +10 after a single handover confirmation. Which service and method contains the update logic?
 5. Scenario: An integration test receives HTTP 400 with `TARGET_NOT_FOUND` when creating a report on a user. What database check failed?
 6. Scenario: Mongoose prints a warning about duplicate indexes on `handoverId`. Which file contains the redundant index declaration?
-

23. Answers to Self-Test

1. `authenticate` middleware (authored by Engineer 1, consumed by Engineer 4).
2. HTTP `401 UNAUTHORIZED`.
3. `Contribution` (`src/models/Contribution.js`).
4. `isValidObjectId` from `src/utils/objectId.js`.
5. Because `Notification.js` explicitly defines custom `createdAt` with `immutable: true` and does not track updates (`updatedAt`).
6. `/api/admin/analytics` in `src/routes/admin.analytics.routes.js`.
7. 500 characters (`isLength({ max: 500 })`).
8. `"open"`.
9. Both `confirmedByProvider` and `confirmedBySeeker` must be `true`.
10. The service returns the existing handover state idempotently without re-executing contributions.
11. HTTP `409 HANDOVER_INACTIVE` ("This handover is no longer active.").
12. To catch concurrent duplicate insert race conditions and safely return the existing record.
13. By falling back to raw MongoDB collection query
`mongoose.connection.collection("resources").findOne({ _id })`.
14. `reviewedBy` (User ObjectId) and `resolution` (non-empty string).
15. `stats.completedTransfers` (+1), `reputationScore` (+10), and `stats.reputationScore` (+10).
16. It queries the raw `contributions` collection using `countDocuments` and sets the user counters to the exact count.
17. The controller ignores the request body and derives `side` by comparing `req.user._id` to `handover.providerId` and `handover.seekerId`. If neither matches, returns 403.
18. No. `transactions.controller.js` explicitly requires the caller to be either `providerId` or `seekerId`; admins are blocked with 403 to prevent fraudulent completion.
19. `acceptanceRate = accepted / (accepted + rejected)` rounded to 4 decimal places.
20. The first `$group` counts items per `(categoryId, status)` pair; the second groups by `categoryId` to compute category totals and conditional status sums (`published` vs `fulfilled`).
21. The controller strictly ignores `req.query.userId` and takes the identity directly from `req.user._id`.
22. By explicitly constructing a new object with only whitelisted, trimmed string properties and casting IDs to `mongoose.Types.ObjectId`.
23. To prevent counter drift and maintain real-time accuracy via MongoDB aggregation pipelines.
24. It makes an invalid request, inspects the JSON response body, and asserts that neither `stack`, `name`, nor raw Mongoose error strings appear in the payload.
25. Inspect lines 28–40 of `src/controllers/transactions.controller.js` where `userId` is compared to `handover.providerId` and `handover.seekerId`.
26. Inspect `aggregateResources` and `countResources` in `src/services/adminAnalyticsService.js` lines 14–37.
27. The `resolution` note was missing or empty (`resolution` is required when setting status to `resolved`).
28. Inspect `recordCompletedTransfer` in `src/services/contributionService.js` lines 78–84.
29. `validateReportTarget` in `src/services/reportService.js` failed to find a user matching the provided `targetId`.

30. `src/models/Contribution.js` at line 16 and line 55.